

Empirical Evidence of Operational Domain Shift

We hypothesize that deployment scenes occupy a different feature distribution than training scenes. Before proposing domain adaptation mechanisms, we first test whether such a shift exists empirically.

Training Domain: Images used during detector training.

Deployment Domain: Frames extracted from video clips captured in completely different settings and different cameras as compared to training domain.

Feature Extraction:

```
from ultralytics import YOLO
from pathlib import Path
import torch
import cv2
import numpy as np
from tqdm import tqdm

model = YOLO("best.pt")

if model.model is None:
    raise RuntimeError("Model failed to load")
else:
    backbone = model.model.model[:10]

def get_embedding(img_path):

    img = cv2.imread(str(img_path))
    img = cv2.resize(img, (640, 640))

    img = img[:, :, ::-1]
    img = img.astype(np.float32) / 255.0

    img = torch.tensor(img).permute(2, 0, 1).unsqueeze(0)

    with torch.no_grad():
```

```

        feat = img

        for layer in backbone:
            feat = layer(feat)

        emb = feat.mean(dim=[2,3])

    return emb.squeeze().cpu().numpy()

train_embeddings = []
deploy_embeddings = []

for p in tqdm(Path("data/train_domain_images/").glob("*")):
    train_embeddings.append(get_embedding(p))

for p in tqdm(Path("data/deployment_domain_images/").glob("*")):
    deploy_embeddings.append(get_embedding(p))

train_embeddings = np.array(train_embeddings)
deploy_embeddings = np.array(deploy_embeddings)

np.save("train_embeddings.npy", train_embeddings)
np.save("deploy_embeddings.npy", deploy_embeddings)

```

1. UMAP Visualisation:

UMAP is a dimensionality-reduction technique that projects high-dimensional feature embeddings into two dimensions while preserving local neighborhood structure. It allows visual inspection of whether samples from different domains occupy similar or different regions of feature space.

Code:

```

import numpy as np
import umap
import matplotlib.pyplot as plt

train = np.load("train_embeddings.npy")
deploy = np.load("deploy_embeddings.npy")

```

```
X = np.vstack([train,deploy])

labels = (
    ["Train"]*len(train)
    +
    ["Deploy"]*len(deploy)
)

embedding = umap.UMAP(
    n_neighbors=15,
    min_dist=0.1,
    random_state=42
).fit_transform(X)

plt.figure(figsize=(8,6))

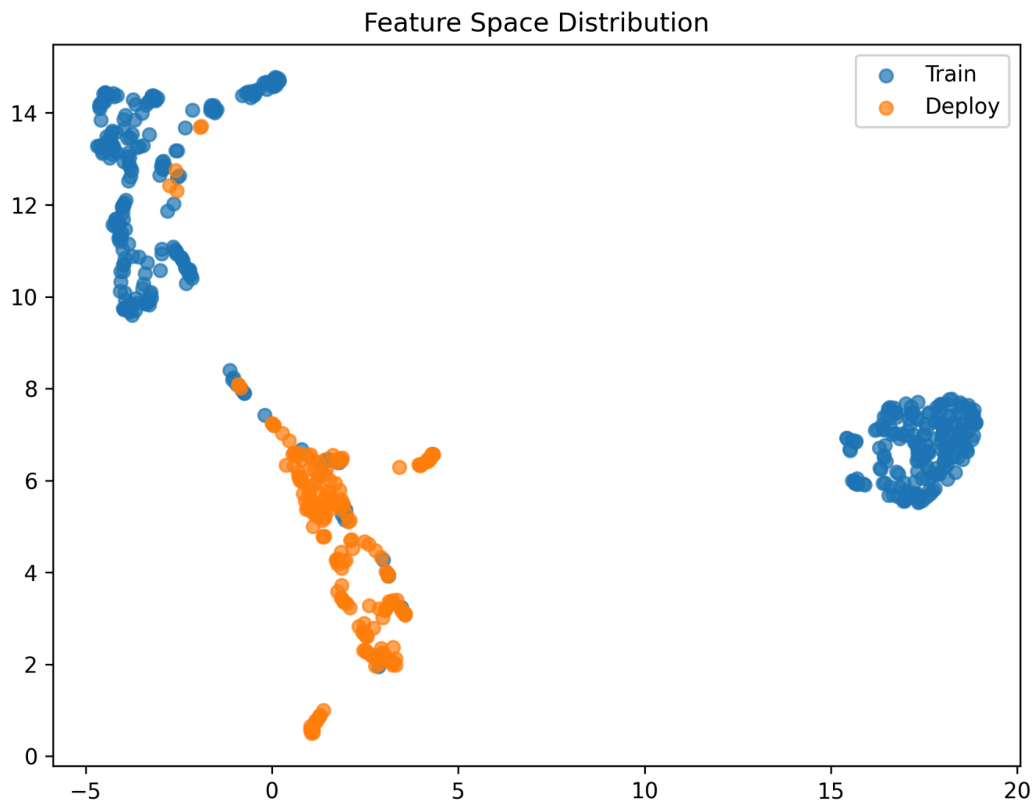
for label in ["Train","Deploy"]:

    idx = [i for i,l in enumerate(labels) if l==label]

    plt.scatter(
        embedding[idx,0],
        embedding[idx,1],
        label=label,
        alpha=0.7
    )

plt.legend()
plt.title("Feature Space Distribution")
plt.savefig("umap_shift.png",dpi=300)
plt.show()
```

Output:



Inference: Deployment samples form a distinct cluster with limited overlap with training samples, suggesting that the detector encodes deployment imagery differently from training imagery.

The backbone embeddings generated from deployment imagery occupy a substantially different region of feature space than embeddings generated from training imagery, suggesting a distributional mismatch between training and deployment conditions.

2. Wasserstein Distance

Visual clustering alone is insufficient. We therefore compute the Wasserstein distance, which measures the minimum transport cost required to transform one feature distribution into another.

Code:

```
import numpy as np
import ot

train = np.load("train_embeddings.npy")
deploy = np.load("deploy_embeddings.npy")

n = len(train)
m = len(deploy)

a = np.ones(n) / n
b = np.ones(m) / m

M = ot.dist(train, deploy)

distance = ot.emd2(a, b, M)

print("Wasserstein Distance =", distance)
```

Output:

Wasserstein Distance = 30.169723320394198

Inference: The Wasserstein distance between training and deployment feature distributions was measured as 30.17, indicating a non-zero transport cost between the two distributions.

3. Domain Classification Experiment

If training and deployment samples originate from the same distribution, a classifier should not be able to distinguish them significantly better than random guessing.

Code:

```
import numpy as np
```

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

train = np.load("train_embeddings.npy")
deploy = np.load("deploy_embeddings.npy")

X = np.vstack([train,deploy])

y = np.array(
    [0]*len(train)
    +
    [1]*len(deploy)
)

X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

clf = RandomForestClassifier()

clf.fit(X_train,y_train)

pred = clf.predict(X_test)

acc = accuracy_score(y_test,pred)

print("Domain Classification Accuracy =",acc)

```

Output:

Domain Classification Accuracy = 93.75%

Inference: The domain classifier achieved 93.75% accuracy when distinguishing training embeddings from deployment embeddings. Since chance performance is 50%, this indicates that the latent representations contain substantial domain-specific information and that the two datasets are not sampled from the same underlying distribution.

4. Conclusion:

The three complementary analyses provide consistent evidence of an operational domain shift. UMAP visualization reveals clear clustering differences between training and deployment embeddings. Wasserstein distance confirms a measurable discrepancy between the two feature distributions. Most importantly, a domain classifier achieves 93.75% accuracy when distinguishing training and deployment samples from latent representations alone. Together, these findings indicate that deployment imagery occupies a different region of feature space than the imagery used during training. These results justify subsequent investigation of domain adaptation, domain generalization, and operational manifold learning techniques.